



Sistema de Gestión Hospitalaria

Proyecto Final – Programación Orientada a Objetos

Humberto Hernandez Nonigo
Matricula: 241653

Docente:

Mtra. Ma. de Lourdes Guardado Bustamante

Institución:

Universidad Autonoma de Ciudad Juarez

Materia:

Programación Orientada a Objetos (POO)

Fecha de entrega:

11 de mayo de 2026

Índice

1. Resumen	3
2. Introducción	4
2.1. Contexto del Problema	4
2.2. Justificación del Desarrollo	4
3. Planteamiento del Problema	5
3.1. Descripción de la Necesidad de un Software de Gestion Hospitalaria	5
3.2. Impacto del Problema	5
4. Marco Teórico	6
4.1. Programación Orientada a Objetos	6
4.2. Clases y Objetos en C++	6
4.3. Constructor y Destructor	6
4.4. Puntero <code>this</code>	6
4.5. Sobrecarga de Operadores	6
4.6. Herencia	7
4.7. Plantillas (Templates)	7
4.8. Polimorfismo y Métodos Virtuales	7
5. Requerimientos del Sistema	8
5.1. Requerimientos de Hardware	8
5.2. Requerimientos de Software	8
6. Diseño del Sistema	10
6.1. Diagrama de Clases (UML)	10
6.2. Diagrama de Secuencia – Programar Cita	11
6.3. Diagrama de Actividad – Flujo Principal	12
7. Implementación	13
7.1. Plantilla Genérica: <code>Repositorio<T></code>	13
7.2. Herencia y Polimorfismo: <code>Persona</code> , <code>Medico</code> , <code>Paciente</code>	13
7.3. Sobrecarga de Operadores en la Clase <code>Cita</code>	14
7.4. Validación de Disponibilidad y Gestión de Citas	15
8. Pruebas	15
8.1. Casos de Prueba	16
8.2. Evidencias de Ejecución	23
8.3. Asistencia en el Desarrollo	24
9. Conclusiones	24

10. Anexos	25
10.1. Repositorio de Código Fuente (GitHub)	25
10.2. Manual de Usuario	25
10.2.1. Instalación y Compilación	25
10.2.2. Uso del Menú	26
10.2.3. Nombres Precargados de Médicos	26
10.3. Nombres Precargados de Pacientes	26
10.4. Conceptos de POO Utilizados	27
11. Referencias y Bibliografía	28

1. Resumen

El presente proyecto implementa un **Sistema de Gestión Hospitalaria** desarrollado en lenguaje C++ por medio de la **Programación Orientada a Objetos** (POO). El sistema permite administrar de manera integral los recursos humanos y operativos de un hospital ficticio: registro de médicos y pacientes, programación y cancelación de citas médicas, validación de disponibilidad de médicos, consulta de historial clínico y generación de reportes en archivos de texto.

La aplicación hace uso explícito de los conceptos fundamentales de POO: clases, constructores, destructores, herencia, polimorfismo, sobrecarga de operadores, plantillas genéricas (*templates*), puntero `this` y mensajes entre objetos. El programa incluye datos precargados de 5 médicos y 15 pacientes, y funciona en un ciclo continuo hasta que el usuario decide salir.

2. Introducción

2.1. Contexto del Problema

Los hospitales y centros de salud son organizaciones altamente complejas donde la correcta gestión de la información es determinante para la calidad de la atención médica. Actualmente, muchos procesos administrativos – como la agenda de citas, la asignación de médicos o el seguimiento de pacientes – se realizan de forma manual o con herramientas desconectadas, lo que genera errores, duplicidades y pérdida de tiempo.

La administración ineficiente de citas médicas puede provocar conflictos de horario entre médicos, tiempos de espera prolongados para los pacientes e imposibilidad de rastrear el historial clínico de forma oportuna.

2.2. Justificación del Desarrollo

La implementación de un sistema de software orientado a objetos para la gestión hospitalaria ofrece una solución estructurada, modular y escalable a estos problemas. Aplicar los principios de POO permite modelar entidades del mundo real (médicos, pacientes, citas) como objetos con atributos y comportamiento propios, facilitando el mantenimiento y la extensión del sistema. El proyecto también sirve como ejercicio académico para aplicar los conceptos estudiados en la materia de Programación Orientada a Objetos.

3. Planteamiento del Problema

3.1. Descripción de la Necesidad de un Software de Gestion Hospitalaria

Un hospital requiere un sistema capaz de:

- Registrar y consultar datos de **médicos** (nombre, especialidad, horario, teléfono) y **pacientes** (nombre, edad, domicilio, teléfono, médico asignado).
- **Programar, cancelar y consultar citas** médicas, garantizando que no existan conflictos de horario para un mismo médico.
- **Ordenar** las citas de cada médico cronológicamente.
- Consultar el **historial completo** de citas de un paciente.
- **Exportar** información a archivos de texto para su distribución.

3.2. Impacto del Problema

Sin un sistema de gestión automatizado:

- Se generan **citas duplicadas** que afectan la atención médica.
- Los médicos desconocen su agenda diaria de manera inmediata.
- Los pacientes no pueden acceder a su historial de consultas de forma eficiente.
- La administración del hospital pierde tiempo valioso en tareas manuales.

4. Marco Teórico

4.1. Programación Orientada a Objetos

La **POO** es un paradigma de programación que organiza el software en torno a *objetos*, que combinan datos (*atributos*) y comportamiento (*métodos*). Sus pilares fundamentales son:

- **Encapsulamiento:** oculta los detalles internos de un objeto y expone solo una interfaz controlada.
- **Herencia:** permite que una clase derive atributos y métodos de otra, promoviendo la reutilización de código.
- **Polimorfismo:** permite que objetos de distintas clases respondan de forma diferente al mismo mensaje (método virtual).
- **Abstracción:** modela solo las características relevantes del mundo real.

4.2. Clases y Objetos en C++

En C++, una **clase** es el molde a partir del cual se crean **objetos**. La clase define los atributos (variables miembro) y los métodos (funciones miembro).

4.3. Constructor y Destructor

El **constructor** es un método especial que se invoca automáticamente al crear un objeto para inicializar sus atributos. El **destructor** se ejecuta cuando el objeto es eliminado, liberando los recursos que haya adquirido.

4.4. Puntero `this`

El puntero **this** es un puntero implícito que apunta al objeto actual dentro de un método. Permite diferenciar atributos propios de parámetros con el mismo nombre y hacer referencia explícita al objeto en curso.

4.5. Sobrecarga de Operadores

C++ permite redefinir el comportamiento de operadores estándar (<, ==, <<, etc.) para tipos definidos por el usuario, lo que mejora la legibilidad y naturalidad del código.

4.6. Herencia

La **herencia** permite crear nuevas clases a partir de clases existentes. En este proyecto, **Medico** y **Paciente** heredan de **Persona**, compartiendo atributos comunes (nombre, edad, domicilio, teléfono) y especializando su comportamiento.

4.7. Plantillas (Templates)

Las **plantillas** permiten escribir código genérico que funciona con diferentes tipos de datos. En este proyecto, la clase **Repositorio<T>** es una plantilla que actúa como contenedor genérico para cualquier tipo de entidad del sistema.

4.8. Polimorfismo y Métodos Virtuales

El método **escritura()** se declara virtual puro en **Persona** y es implementado de forma diferente por **Medico** y **Paciente**, permitiendo que el mismo mensaje produzca respuestas distintas según el tipo de objeto.

5. Requerimientos del Sistema

5.1. Requerimientos de Hardware

Cuadro 1: Requerimientos mínimos de hardware

Componente	Especificación mínima
Procesador	Intel Core i3 o equivalente (1.5 GHz)
Memoria RAM	512 MB
Almacenamiento	50 MB de espacio libre
Pantalla	Resolución mínima 800×600

5.2. Requerimientos de Software

Cuadro 2: Requerimientos de software

Software	Versión
Sistema Operativo	Windows 10, Linux (Ubuntu 20.04+) o macOS 11+
Compilador	g++ (GCC 9.0+) con soporte para C++17
IDE (opcional)	Visual Studio Code, Code::Blocks, CLion
Terminal	CMD, PowerShell, Bash o Zsh



Sistema operativo	Linux Mint 22.2 Cinnamon
Versión de Cinnamon	6.4.8
Núcleo Linux	6.17.0-23-generic
Procesador	Intel® Core™ i7-4600U CPU @ 2.10GHz × 2
Memoria	7.6 GiB
Disco duro	240.1 GB
Tarjeta gráfica	Intel Corporation Haswell-ULT Integrated Graphics Controller
Servidor gráfico	X11
Subir la información del sistema	
Copiar al portapapeles	

Figura 1: Equipo utilizado para el desarrollo de este programa.

6. Diseño del Sistema

6.1. Diagrama de Clases (UML)

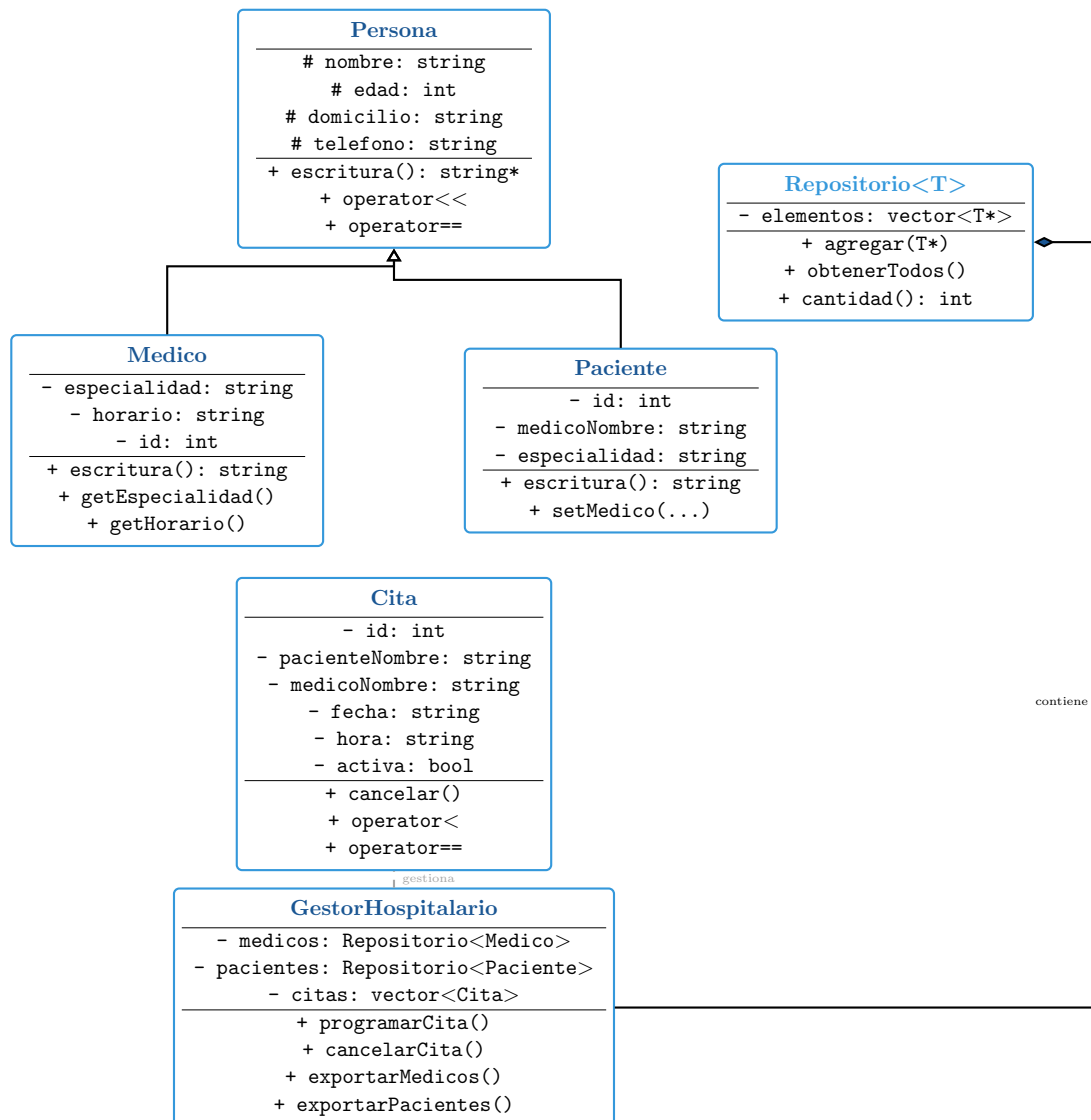


Figura 2: Diagrama de Clases UML del Sistema de Gestión Hospitalaria

6.2. Diagrama de Secuencia – Programar Cita

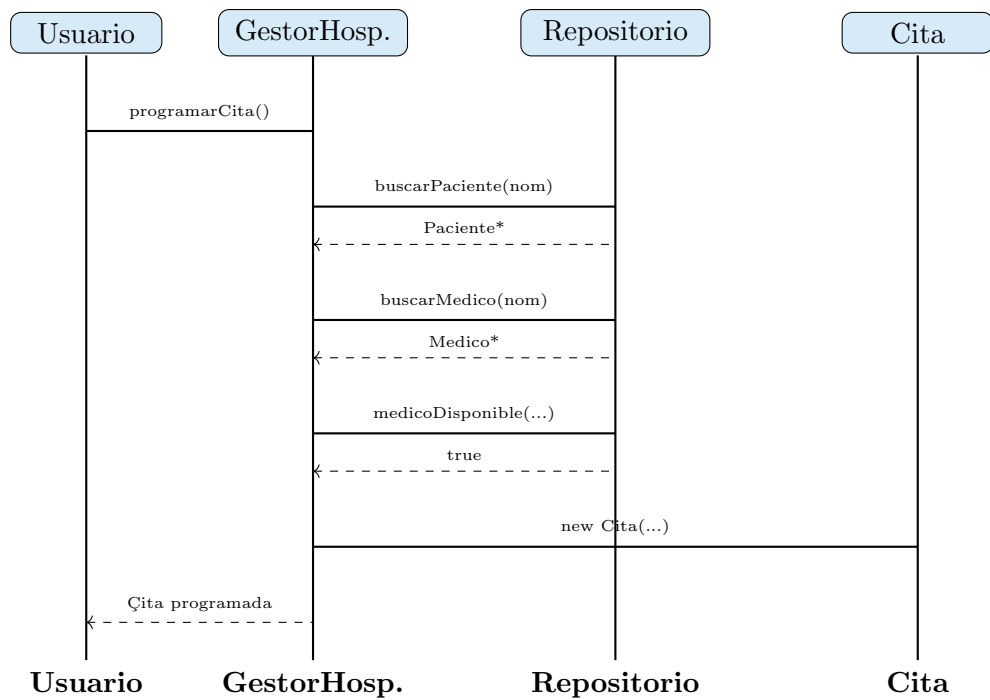


Figura 3: Diagrama de Secuencia: Programar una Cita Médica

6.3. Diagrama de Actividad – Flujo Principal

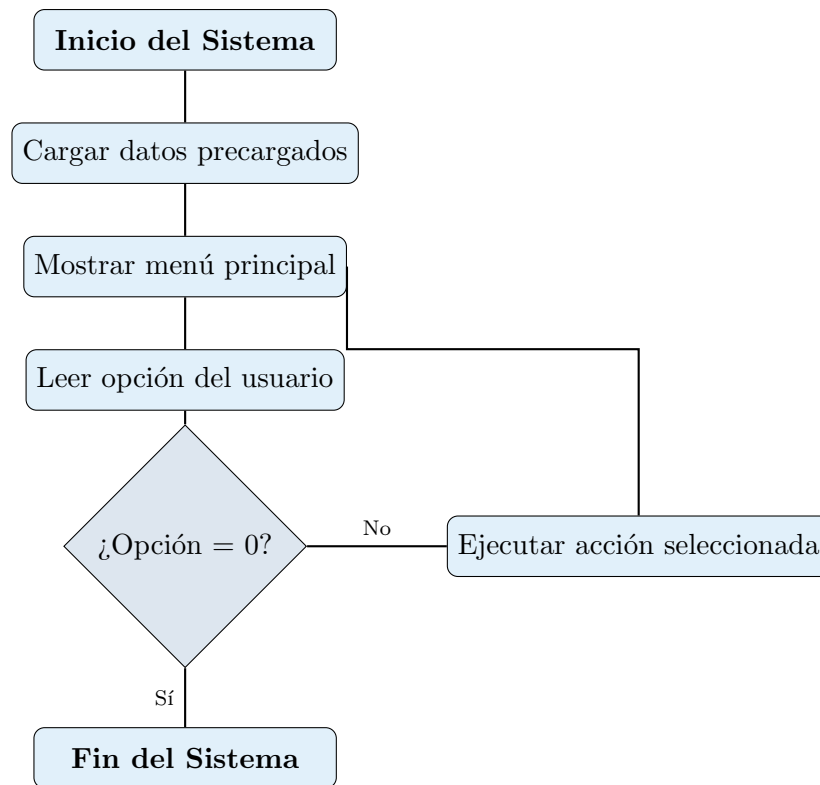


Figura 4: Diagrama de Actividad: Flujo General del Sistema

7. Implementación

7.1. Plantilla Genérica: Repositorio<T>

La plantilla permite reutilizar el mismo código de contenedor para médicos y pacientes.

```
1  template <typename T>
2  class Repositorio {
3  private:
4      vector<T*> elementos; // Vector de punteros al tipo generico T
5  public:
6      ~Repositorio() {
7          for (T* e : elementos) delete e; // Libera cada objeto
8              dinamico
9          elementos.clear();
10     }
11     void agregar(T* elem) { elementos.push_back(elem); }
12     const vector<T*>& obtenerTodos() const { return elementos; }
13     int cantidad() const { return (int)elementos.size(); }
14 };
15
```

Listing 1: Plantilla Repositorio

7.2. Herencia y Polimorfismo: Persona, Medico, Paciente

```
1  class Persona {
2  protected:
3      string nombre, domicilio, telefono;
4      int edad;
5  public:
6      Persona(const string& n, int e, const string& d, const string& t
7          )
8          : nombre(n), edad(e), domicilio(d), telefono(t) {}
9      virtual ~Persona() {}
10     // Método virtual puro: obliga a subclases a implementarlo
11     virtual string escritura() const = 0;
12     // Sobrecarga de operador <<
13     friend ostream& operator<<(ostream& os, const Persona& p) {
14         os << p.escritura(); return os;
15     }
16 };
17
```

Listing 2: Clase base Persona con método virtual puro

```
1 class Medico : public Persona {
2     string especialidad, horario;
3     int id;
4     static int contadorId;
5 public:
6     Medico(const string& n, int e, const string& d, const string& t,
7           const string& esp, const string& hor)
8         : Persona(n,e,d,t), especialidad(esp), horario(hor) {
9         id = ++contadorId;
10    }
11    string escritura() const override {
12        ostringstream oss;
13        // Uso explícito del puntero this
14        oss << "[Medico #" << this->id << "] " << this->nombre
15            << " | " << this->especialidad << " | " << this->horario
16            ;
17        return oss.str();
18    };
19 }
```

Listing 3: Clase Medico con uso de puntero this

7.3. Sobrecarga de Operadores en la Clase Cita

```
1 // Operador < para ordenar citas cronologicamente
2 bool operator<(const Cita& otra) const {
3     if (this->fecha != otra.fecha) return this->fecha < otra.fecha;
4     return this->hora < otra.hora;
5 }
6 // Operador == para detectar citas duplicadas
7 bool operator==(const Cita& otra) const {
8     return medicoNombre == otra.medicoNombre &&
9           fecha           == otra.fecha           &&
10          hora             == otra.hora;
11 }
```

Listing 4: Sobrecarga de operadores en Cita

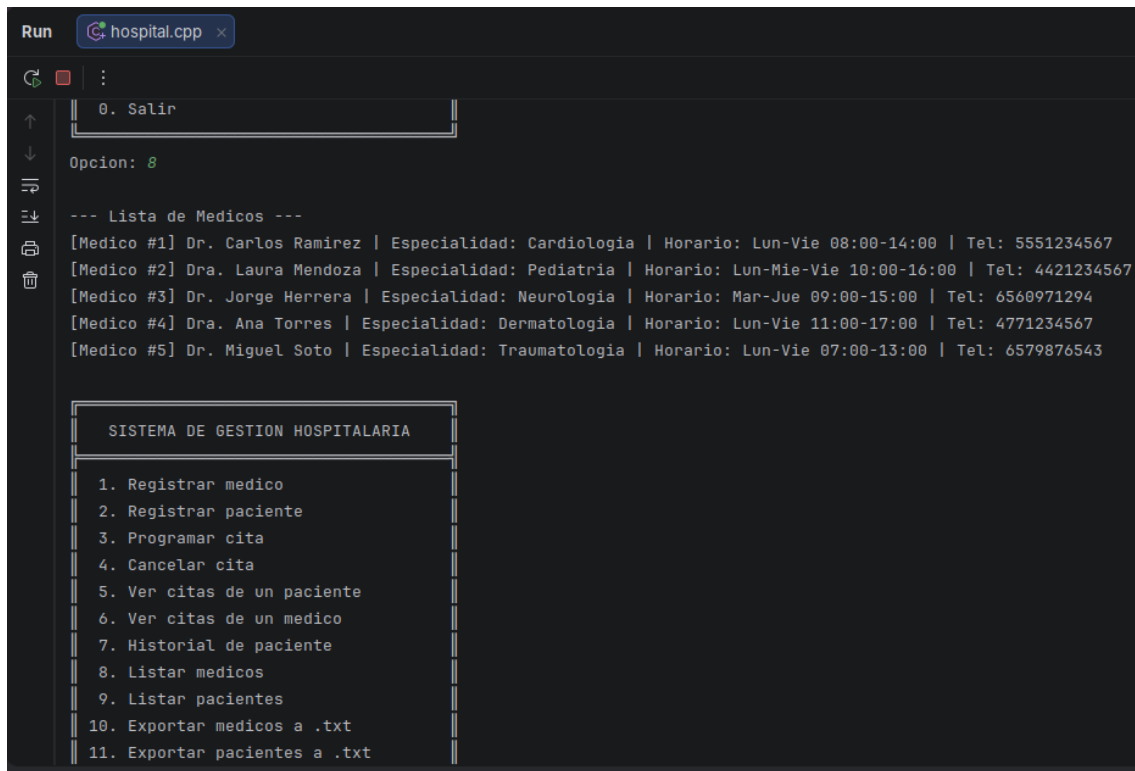
7.4. Validación de Disponibilidad y Gestión de Citas

```
1 bool medicoDisponible(const string& medNombre,
2                       const string& fecha,
3                       const string& hora) const {
4     for (const Cita& c : citas) {
5         if (c.getMedicoNombre() == medNombre &&
6             c.getFecha() == fecha &&
7             c.getHora() == hora &&
8             c.estaActiva())
9         {
10             return false; // Ya ocupado en ese horario
11         }
12     }
13     return true;
14 }
```

Listing 5: Validación de disponibilidad del médico

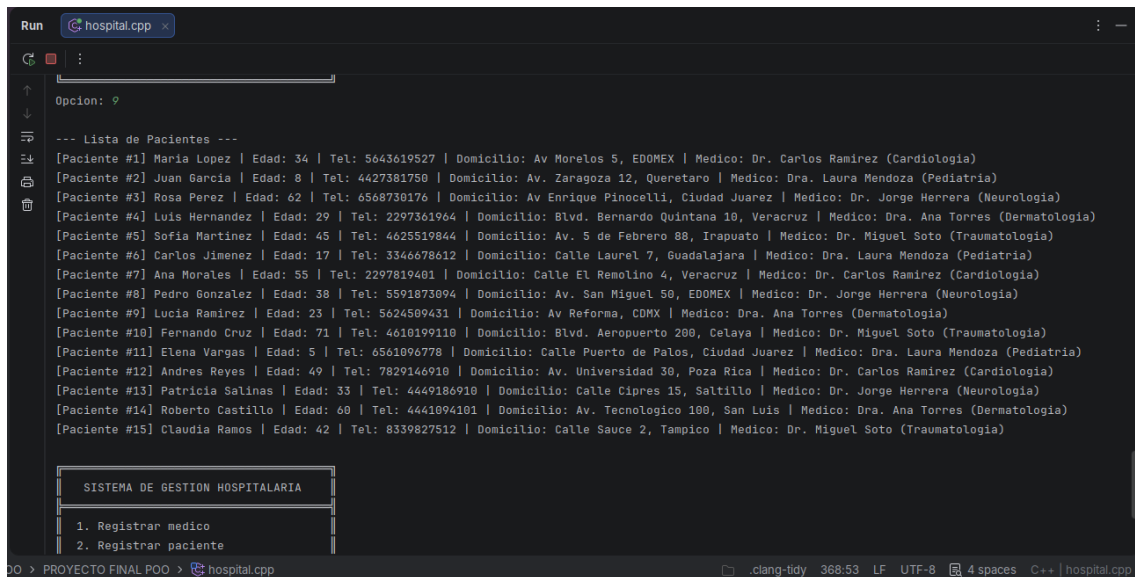
8. Pruebas

8.1. Casos de Prueba



```
Run hospital.cpp x
0. Salir
Opcion: 8
--- Lista de Medicos ---
[Medico #1] Dr. Carlos Ramirez | Especialidad: Cardiologia | Horario: Lun-Vie 08:00-14:00 | Tel: 5551234567
[Medico #2] Dra. Laura Mendoza | Especialidad: Pediatria | Horario: Lun-Mie-Vie 10:00-16:00 | Tel: 4421234567
[Medico #3] Dr. Jorge Herrera | Especialidad: Neurologia | Horario: Mar-Jue 09:00-15:00 | Tel: 6560971294
[Medico #4] Dra. Ana Torres | Especialidad: Dermatologia | Horario: Lun-Vie 11:00-17:00 | Tel: 4771234567
[Medico #5] Dr. Miguel Soto | Especialidad: Traumatologia | Horario: Lun-Vie 07:00-13:00 | Tel: 6579876543
SISTEMA DE GESTION HOSPITALARIA
1. Registrar medico
2. Registrar paciente
3. Programar cita
4. Cancelar cita
5. Ver citas de un paciente
6. Ver citas de un medico
7. Historial de paciente
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
```

Figura 5: Medicos Pre-cargados



```
Run hospital.cpp x
Opcion: 2
--- Lista de Pacientes ---
[Paciente #1] Maria Lopez | Edad: 34 | Tel: 5643619527 | Domicilio: Av Morelos 5, EDOMEX | Medico: Dr. Carlos Ramirez (Cardiologia)
[Paciente #2] Juan Garcia | Edad: 8 | Tel: 4427381750 | Domicilio: Av. Zaragoza 12, Queretaro | Medico: Dra. Laura Mendoza (Pediatria)
[Paciente #3] Rosa Perez | Edad: 62 | Tel: 6568730176 | Domicilio: Av Enrique Pinocelli, Ciudad Juarez | Medico: Dr. Jorge Herrera (Neurologia)
[Paciente #4] Luis Hernandez | Edad: 29 | Tel: 2297361964 | Domicilio: Blvd. Bernardo Quintana 10, Veracruz | Medico: Dra. Ana Torres (Dermatologia)
[Paciente #5] Sofia Martinez | Edad: 45 | Tel: 4625519844 | Domicilio: Av. 5 de Febrero 88, Irapuato | Medico: Dr. Miguel Soto (Traumatologia)
[Paciente #6] Carlos Jimenez | Edad: 17 | Tel: 3346678612 | Domicilio: Calle Laurel 7, Guadalajara | Medico: Dra. Laura Mendoza (Pediatria)
[Paciente #7] Ana Morales | Edad: 55 | Tel: 2297819401 | Domicilio: Calle El Remolino 4, Veracruz | Medico: Dr. Carlos Ramirez (Cardiologia)
[Paciente #8] Pedro Gonzalez | Edad: 38 | Tel: 5591873094 | Domicilio: Av. San Miguel 50, EDOMEX | Medico: Dr. Jorge Herrera (Neurologia)
[Paciente #9] Lucia Ramirez | Edad: 23 | Tel: 5624509431 | Domicilio: Av Reforma, CDMX | Medico: Dra. Ana Torres (Dermatologia)
[Paciente #10] Fernando Cruz | Edad: 71 | Tel: 4618199110 | Domicilio: Blvd. Aeropuerto 200, Celaya | Medico: Dr. Miguel Soto (Traumatologia)
[Paciente #11] Elena Vargas | Edad: 5 | Tel: 6561896778 | Domicilio: Calle Puerto de Palos, Ciudad Juarez | Medico: Dra. Laura Mendoza (Pediatria)
[Paciente #12] Andres Reyes | Edad: 49 | Tel: 7829146910 | Domicilio: Av. Universidad 30, Poza Rica | Medico: Dr. Carlos Ramirez (Cardiologia)
[Paciente #13] Patricia Salinas | Edad: 33 | Tel: 4449186910 | Domicilio: Calle Cipres 15, Saltillo | Medico: Dr. Jorge Herrera (Neurologia)
[Paciente #14] Roberto Castillo | Edad: 60 | Tel: 4441094101 | Domicilio: Av. Tecnologico 100, San Luis | Medico: Dra. Ana Torres (Dermatologia)
[Paciente #15] Claudia Ramos | Edad: 42 | Tel: 8339827512 | Domicilio: Calle Sauce 2, Tampico | Medico: Dr. Miguel Soto (Traumatologia)
SISTEMA DE GESTION HOSPITALARIA
1. Registrar medico
2. Registrar paciente
```

Figura 6: Pacientes Pre-cargados

```
Run hospital.cpp x
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
0. Salir

Opcion: 3

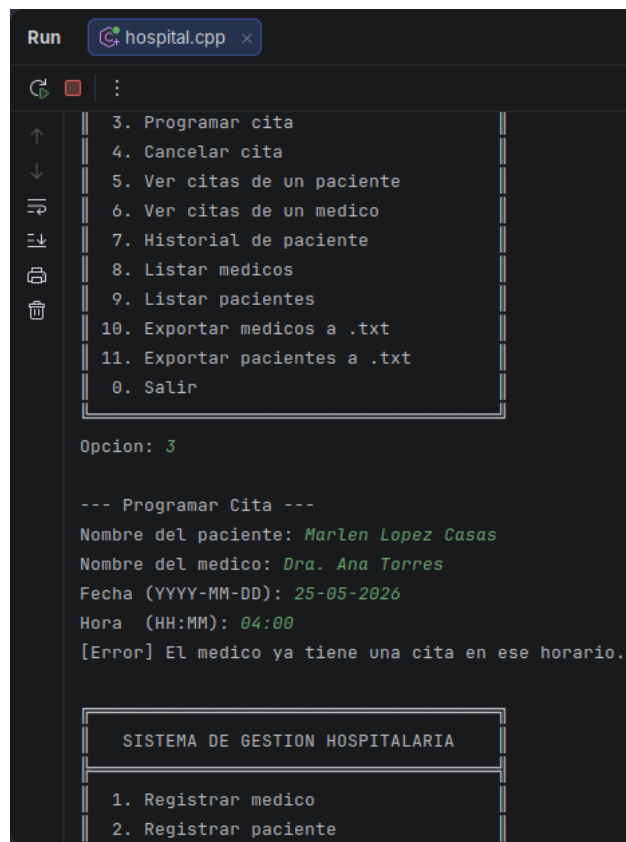
--- Programar Cita ---
Nombre del paciente: Marlen Lopez Casas
Nombre del medico: Dra. Ana Torres
Fecha (YYYY-MM-DD): 25-05-2026
Hora (HH:MM): 04:00

Construccion de la Clase Cita
[OK] Cita programada exitosamente.

SISTEMA DE GESTION HOSPITALARIA

1. Registrar medico
2. Registrar paciente
3. Programar cita
4. Cancelar cita
5. Ver citas de un paciente
```

Figura 7: Programar cita valida



```
Run hospital.cpp x
:
3. Programar cita
4. Cancelar cita
5. Ver citas de un paciente
6. Ver citas de un medico
7. Historial de paciente
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
0. Salir

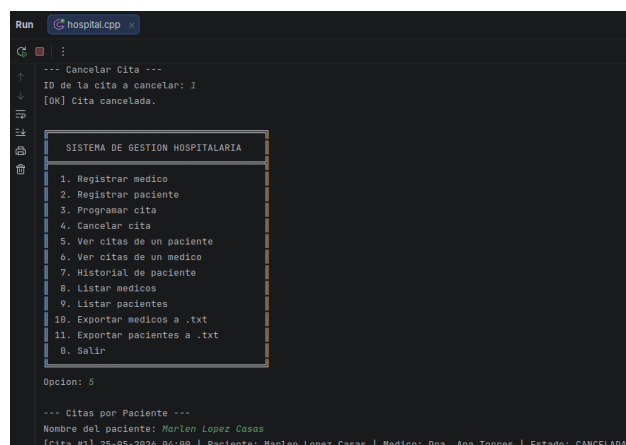
Opcion: 3

--- Programar Cita ---
Nombre del paciente: Marlen Lopez Casas
Nombre del medico: Dra. Ana Torres
Fecha (YYYY-MM-DD): 25-05-2026
Hora (HH:MM): 04:00
[Error] El medico ya tiene una cita en ese horario.

SISTEMA DE GESTION HOSPITALARIA

1. Registrar medico
2. Registrar paciente
```

Figura 8: Programar cita con medico ocupado



```
Run hospital.cpp x
:
--- Cancelar Cita ---
ID de la cita a cancelar: 1
[OK] Cita cancelada.

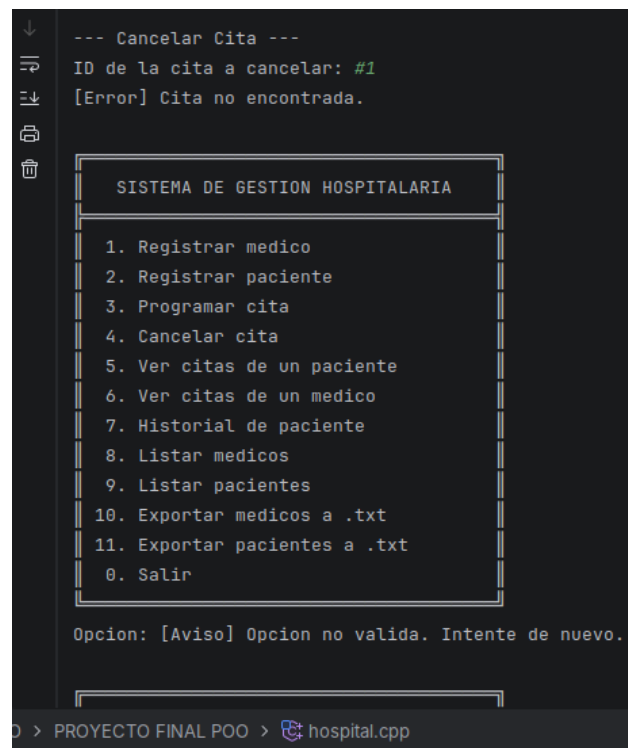
SISTEMA DE GESTION HOSPITALARIA

1. Registrar medico
2. Registrar paciente
3. Programar cita
4. Cancelar cita
5. Ver citas de un paciente
6. Ver citas de un medico
7. Historial de paciente
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
0. Salir

Opcion: 5

--- Citas por Paciente ---
Nombre del paciente: Marlen Lopez Casas
[Cita #1] 25-05-2026 04:00 | Paciente: Marlen Lopez Casas | Medico: Dra. Ana Torres | Estado: CANCELADA
```

Figura 9: Cancelar cita existente



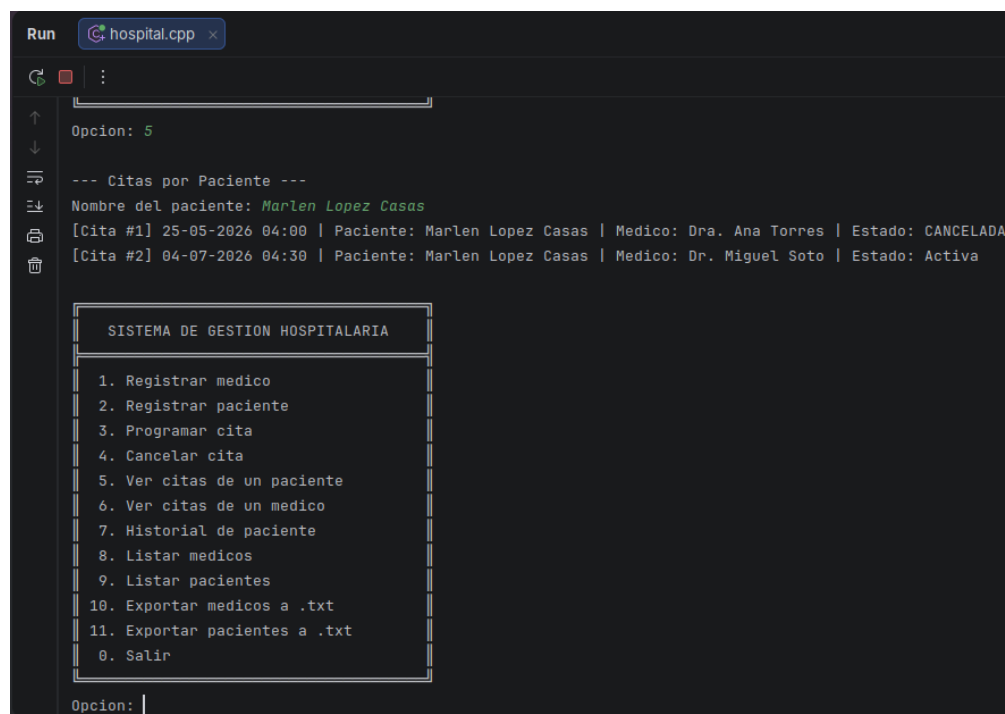
```
--- Cancelar Cita ---
ID de la cita a cancelar: #1
[Error] Cita no encontrada.

SISTEMA DE GESTION HOSPITALARIA

1. Registrar medico
2. Registrar paciente
3. Programar cita
4. Cancelar cita
5. Ver citas de un paciente
6. Ver citas de un medico
7. Historial de paciente
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
0. Salir

Opcion: [Aviso] Opcion no valida. Intente de nuevo.
```

Figura 10: Cancelar cita inexistente



```
Opcion: 5

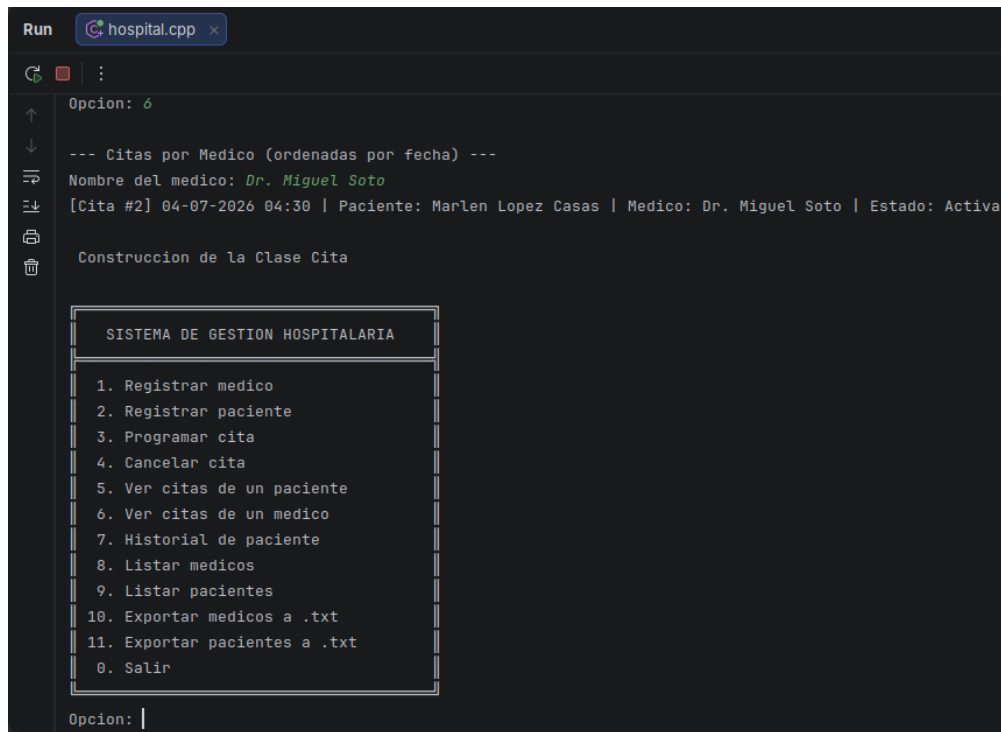
--- Citas por Paciente ---
Nombre del paciente: Marlen Lopez Casas
[Cita #1] 25-05-2026 04:00 | Paciente: Marlen Lopez Casas | Medico: Dra. Ana Torres | Estado: CANCELADA
[Cita #2] 04-07-2026 04:30 | Paciente: Marlen Lopez Casas | Medico: Dr. Miguel Soto | Estado: Activa

SISTEMA DE GESTION HOSPITALARIA

1. Registrar medico
2. Registrar paciente
3. Programar cita
4. Cancelar cita
5. Ver citas de un paciente
6. Ver citas de un medico
7. Historial de paciente
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
0. Salir

Opcion: |
```

Figura 11: Consultar citas de un paciente



```
Run hospital.cpp x
Opcion: 6

--- Citas por Medico (ordenadas por fecha) ---
Nombre del medico: Dr. Miguel Soto
[Cita #2] 04-07-2026 04:30 | Paciente: Marlen Lopez Casas | Medico: Dr. Miguel Soto | Estado: Activa

Construccion de la Clase Cita

SISTEMA DE GESTION HOSPITALARIA

1. Registrar medico
2. Registrar paciente
3. Programar cita
4. Cancelar cita
5. Ver citas de un paciente
6. Ver citas de un medico
7. Historial de paciente
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
0. Salir

Opcion: |
```

Figura 12: Citas de medico ordenadas



```
Run hospital.cpp x
6. Ver citas de un medico
7. Historial de paciente
8. Listar medicos
9. Listar pacientes
10. Exportar medicos a .txt
11. Exportar pacientes a .txt
0. Salir

Opcion: 7

--- Historial de Paciente ---
Nombre del paciente: Marlen Lopez Casas

Datos del paciente:
[Paciente #16] Marlen Lopez Casas | Edad: 23 | Tel: 6569318321 | Domicilio: Senderos de San Isidro, Ciudad Juarez | Medico: Sin asignar (N/A)

Historial de citas:
[Cita #1] 25-05-2026 04:00 | Paciente: Marlen Lopez Casas | Medico: Dra. Ana Torres | Estado: CANCELADA
[Cita #2] 04-07-2026 04:30 | Paciente: Marlen Lopez Casas | Medico: Dr. Miguel Soto | Estado: Activa

SISTEMA DE GESTION HOSPITALARIA

1. Registrar medico
2. Registrar paciente
```

Figura 13: Historial medico de un paciente

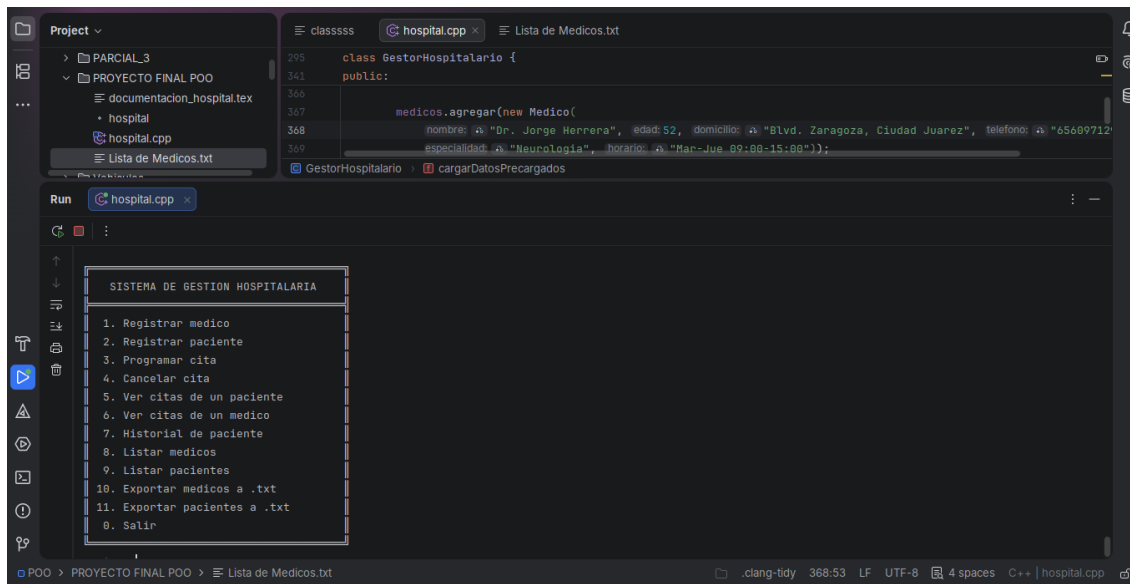


Figura 14: Exportar medicos a .txt

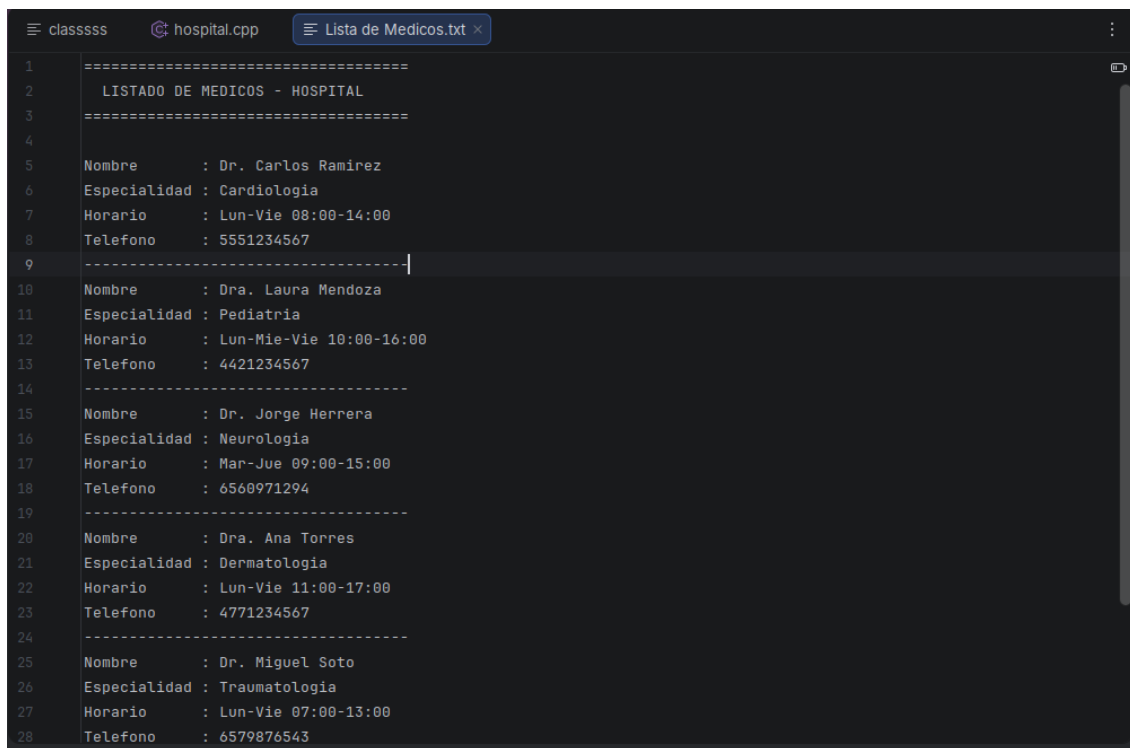


Figura 15: Listado de Medicos en el Hospital

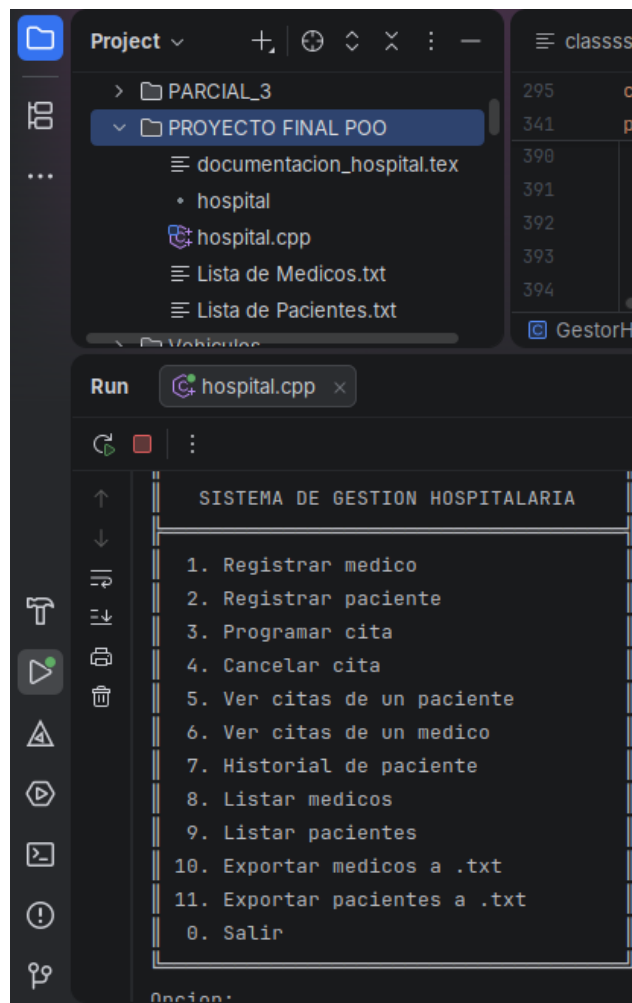
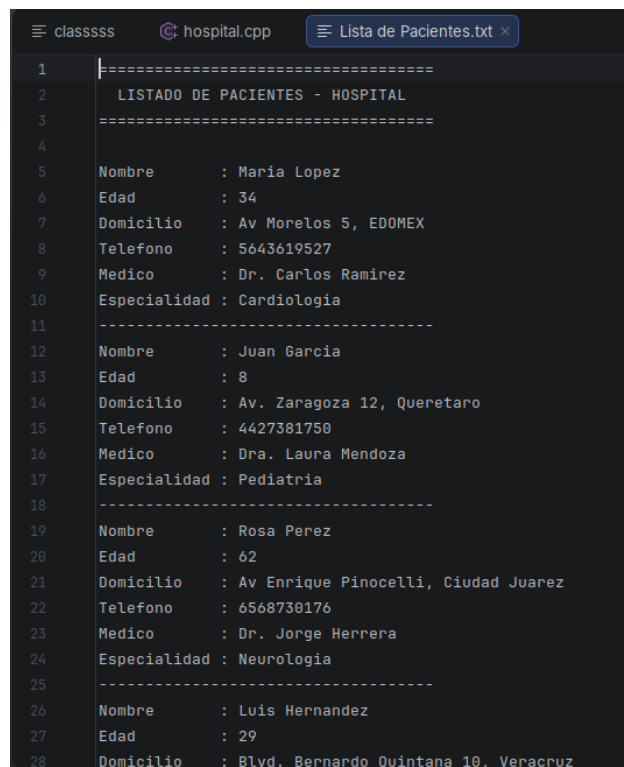


Figura 16: Exportar pacientes a .txt



The screenshot shows a code editor with a dark theme. The top bar indicates the file is 'hospital.cpp' and the current view is 'Lista de Pacientes.txt'. The code is a C++ program that prints a list of patients. The output is as follows:

```
1 =====
2 LISTADO DE PACIENTES - HOSPITAL
3 =====
4
5 Nombre      : Maria Lopez
6 Edad       : 34
7 Domicilio  : Av Morelos 5, EDOMEX
8 Telefono    : 5643619527
9 Medico     : Dr. Carlos Ramirez
10 Especialidad : Cardiologia
11 -----
12 Nombre     : Juan Garcia
13 Edad      : 8
14 Domicilio  : Av. Zaragoza 12, Queretaro
15 Telefono   : 4427381750
16 Medico     : Dra. Laura Mendoza
17 Especialidad : Pediatría
18 -----
19 Nombre     : Rosa Perez
20 Edad      : 62
21 Domicilio  : Av Enrique Pinocelli, Ciudad Juarez
22 Telefono   : 6568730176
23 Medico     : Dr. Jorge Herrera
24 Especialidad : Neurologia
25 -----
26 Nombre     : Luis Hernandez
27 Edad      : 29
28 Domicilio  : Blvd. Bernardo Quintana 10, Veracruz
```

Figura 17: Listado de Pacientes

8.2. Evidencias de Ejecución

Para compilar y ejecutar el sistema, se usa el siguiente comando en la terminal:

```
1 g++ -std=c++17 -o hospital hospital.cpp
2 ./hospital
```

Listing 6: Compilación y ejecución

Al iniciar, el sistema muestra los datos precargados y el menú principal. A continuación se describe la salida esperada:

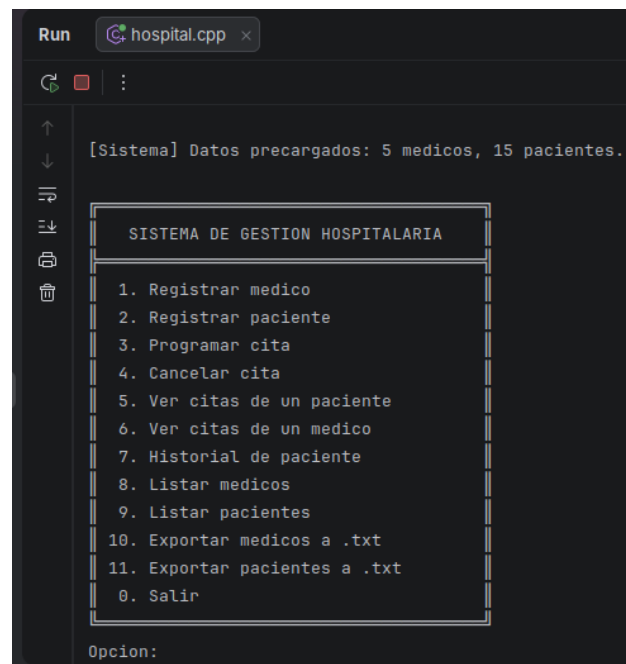


Figura 18: Interfaz del usuario con visibilidad del menu disponible

8.3. Asistencia en el Desarrollo

Se reconoce la asistencia de la Inteligencia Artificial Gemini (Google) como herramienta de apoyo didáctico. Su intervención fue clave para la optimización de la lógica de persistencia, específicamente en la integración de los métodos de carga de ficheros (`cargarMedicosDesdeTxt` y `cargarPacientesDesdeTxt`). Esto permitió que el sistema, además de manejar datos precargados, sea capaz de recuperar y reconstruir objetos guardados en sesiones anteriores, garantizando la continuidad de la información.

9. Conclusiones

El desarrollo del Sistema de Gestión Hospitalaria me permitió aplicar de forma práctica e integral los conceptos fundamentales de la Programación Orientada a Objetos en C++. A lo largo del proyecto se evidenció cómo la herencia —a través de la jerarquía **Persona** → **Medico** / **Paciente**— permite reducir la duplicación de código y modelar relaciones naturales del mundo real. La sobrecarga de operadores fue una gran mejora para la legibilidad del programa, permitiendo comparar y ordenar citas de forma intuitiva. El uso de plantillas genéricas demostró cómo una sola estructura de datos puede servir para diferentes tipos de objetos sin arriesgar la seguridad de tipos.

El puntero `this` resultó ser muy fundamental para diferenciar atributos propios de parámetros

en contextos de gran similitud, y los constructores y destructores garantizaron una gestión correcta de la memoria dinámica (cabe recalcar que se muestra el mensaje de su llamada para entender mejor cuando un objeto se crea y se destruye). La validación de disponibilidad de médicos y el ordenamiento de citas son funcionalidades que tienen un impacto directo en la eficiencia de la atención hospitalaria.

Como aprendizaje personal, he de reconocer que el diseño previo de la arquitectura de clases —mediante diagramas UML— es un paso crítico que ahorra tiempo durante la implementación y reduce la probabilidad de errores de diseño. Con base a esto es la primera vez que implemento un diagrama de este tipo y me ayudo a eficientizar mas mi tiempo de desarrollo del programa. En una versión futura del sistema se podría incorporar persistencia real en base de datos, una interfaz gráfica y autenticación de usuarios para un ambiente de producción.

10. Anexos

10.1. Repositorio de Código Fuente (GitHub)

Como parte de la entrega integral del proyecto, el código fuente completo, incluyendo la lógica de las clases, la implementación de la plantilla `Repositorio<T>` y las funciones de persistencia de datos, se encuentra alojado en un repositorio público de GitHub. Esto permite la revisión técnica de la estructura del software y su historial de desarrollo.

- **Descripción:** Sistema de Gestión Hospitalaria en C++ (POO).
- **Enlace al repositorio:** https://github.com/Humberto-55/Sistema_Gestion_Hospitalaria

Nota: El repositorio contiene el archivo `hospital.cpp` y los archivos de datos precargados.

10.2. Manual de Usuario

10.2.1. Instalación y Compilación

1. Asegurarse de tener instalado g++ (GCC 9.0 o superior).
2. Colocar el archivo `hospital.cpp` en una carpeta de trabajo.
3. Abrir la terminal y navegar a la carpeta:

```
1 cd /ruta/al/proyecto
```

4. Compilar el programa:

```
1 g++ -std=c++17 -o hospital hospital.cpp
```

5. Ejecutar:

```
1  ./hospital          # Linux / macOS
2  hospital.exe        # Windows
```

10.2.2. Uso del Menú

Al ejecutar el programa, se presenta el menú con 12 opciones numeradas del 0 al 11. El usuario ingresa el número de la opción deseada y presiona **Enter**. El sistema valida la entrada y, en caso de datos incorrectos, muestra un mensaje de error sin terminar el programa.

Opciones 1–2 Registro interactivo de médicos y pacientes. El sistema solicita los datos uno por uno.

Opciones 3–4 Programación y cancelación de citas. Para programar se necesita el nombre exacto del paciente y del médico. Para cancelar, el ID de la cita (visible al listar citas).

Opciones 5–7 Consultas. Se ingresa el nombre del paciente o médico y el sistema muestra la información correspondiente.

Opciones 8–9 Listado completo de médicos y pacientes en pantalla.

Opciones 10–11 Exportación a archivo `.txt`. El sistema solicita el nombre del archivo (sin extensión) y lo crea en el directorio de ejecución.

Opción 0 Termina el programa de forma limpia, ejecutando todos los destructores.

10.2.3. Nombres Precargados de Médicos

- Dr. Carlos Ramirez – Cardiología
- Dra. Laura Mendoza – Pediatría
- Dr. Jorge Herrera – Neurología
- Dra. Ana Torres – Dermatología
- Dr. Miguel Soto – Traumatología

10.3. Nombres Precargados de Pacientes

- Maria Lopez
- Juan Garcia
- Rosa Perez

- Luis Hernandez
- Sofia Martinez
- Carlos Jimenez
- Ana Morales
- Pedro Gonzalez
- Lucia Ramirez
- Fernando Cruz
- Elena Vargas
- Andres Reyes
- Patricia Salinas
- Roberto Castillo
- Claudia Ramos

10.4. Conceptos de POO Utilizados

Cuadro 3: Conceptos de POO aplicados en el proyecto

Concepto	Aplicación en el proyecto
Clase	Persona, Medico, Paciente, Cita, GestorHospitalario
Constructor	Cada clase inicializa atributos con lista de inicialización
Destructor	Repositorio<T> libera memoria dinámica de todos sus elementos
Herencia	Medico y Paciente heredan de Persona
Polimorfismo	Método virtual puro escritura() en Persona
Puntero this	Acceso explícito en Medico::escritura() y Paciente::setMedico()
Sobrecarga de operadores	<< en Persona; < y == en Cita
Plantillas (templates)	Repositorio<T> funciona con Medico y Paciente
Mensajes entre objetos	GestorHospitalario envía mensajes a Medico, Paciente y Cita

11. Referencias y Bibliografía

Cuadro 4: Fuentes de consulta y recursos utilizados

Fuente / Autor	Descripción y Enlace
Contigo Sin Fronteras	<i>Tabla de Ladas de México para números telefónicos.</i> https://contigosinfronteras.org/wp-content/uploads/2020/02/tabla_ladas.pdf
Deitel, H. & Deitel, P.	<i>C++ Como Programar</i> , 4ta Edición. Pearson/Prentice Hall. (Referencia para C/C++ y Java).
Grätzer, George	<i>Practical L^AT_EX</i> . Springer Cham. https://doi.org/10.1007/978-3-031-55281-6
GeeksforGeeks (2026)	<i>Object Oriented Programming in C++.</i> https://www.geeksforgeeks.org/cpp/object-oriented-programming-in-cpp/
W3Schools	<i>C++ Templates.</i> https://www.w3schools.com/cpp/cpp_templates.asp
Salvador Pozo	<i>Curso C++: Trabajar con ficheros.</i> https://conclase.net/c/curso/cap39
Google Gemini	<i>Asistente de IA: Apoyo en lógica de persistencia de objetos y depuración de diagramas TikZ en C++/LaTeX.</i>